AdaptaBrasil · Mapa de Indicadores

Mapa do Brasil inteiro sem espera

Como o aplicativo de testes passou a exibir os 468 mil setores censitários do país em poucos segundos — e os mapas de estado e município sem falhas entre polígonos: tiles vetoriais gerados pelo PostGIS para todos os mapas, limites de estados e municípios e cores pré-calculadas por indicador.

Repositório: github.com/AdaptaBrasil/webtests · Novembro de 2026

Resumo

Originalmente, um mapa de estado enviava ao navegador todos os polígonos (GeoJSON) e um valor por polígono. Para o país inteiro isso não funciona: são **468.097 setores censitários** com **1,2 GB de geometria**, e ler um indicador para o Brasil todo exige percorrer a tabela `value` inteira (~200 s). A solução separa o problema em duas partes, ambas calculadas uma única vez e guardadas no banco:

- **Geometria em tiles vetoriais** (PostGIS `ST_AsMVT`): o navegador baixa só os pedaços do mapa que estão na tela, já simplificados para o nível de zoom.
- **Cores como uma "string de classes"** por indicador: um caractere por setor com o índice da faixa da legenda — cerca de **8 KB** comprimidos para o país inteiro.

Depois, a mesma camada de tiles passou a ser usada também nos mapas de **estado e município** (que agora só baixam os valores), e uma segunda camada de tiles desenha os **limites de estados e municípios**. Isso eliminou as falhas ("slivers") entre setores vizinhos e reduziu a carga de Minas Gerais de 45,5 s para cerca de 3 s.

> **Resultado:** escolhendo **Brasil** no campo Estado, o mapa nacional aparece em cerca de 5–8 segundos (a maior parte é o navegador aplicando as cores); trocar de indicador custa um download de 8 KB; aproximar o zoom carrega apenas os tiles da área visível. Um estado grande como MG (51 mil setores) abre em 3–4 s; um município como Belo Horizonte, em 0,6 s.

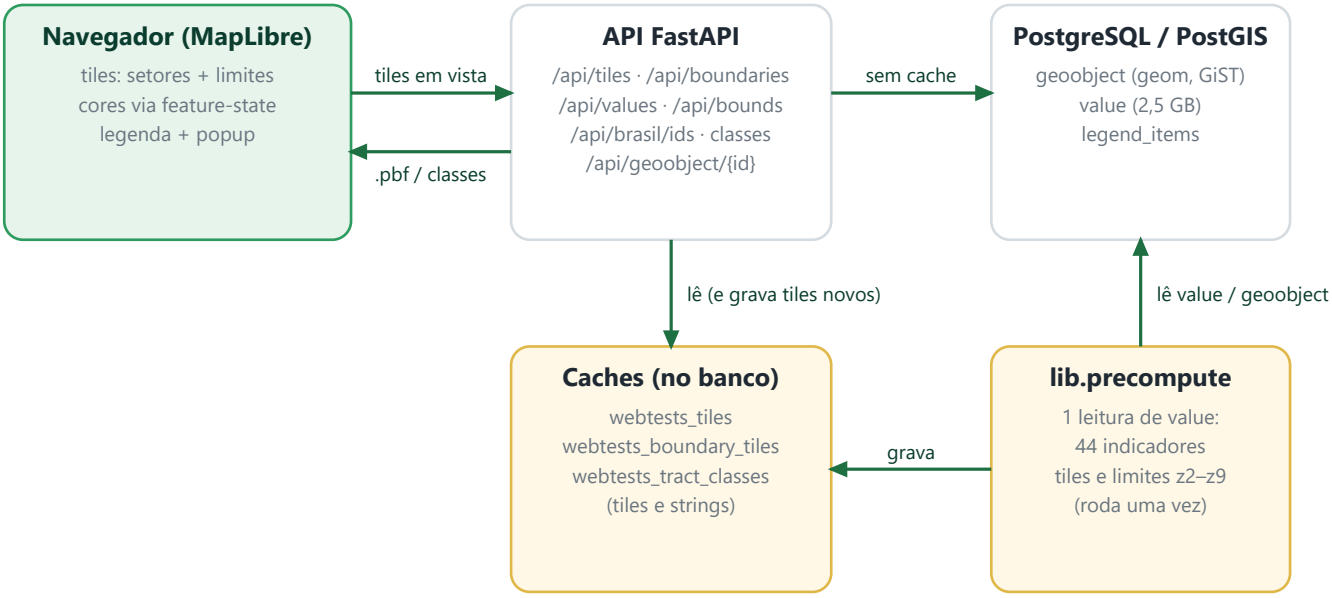
1. O problema

O aplicativo já mostrava estados pequenos e municípios rapidamente. Para o país, os números são de outra ordem:

Item	Tamanho	Consequência
Setores censitários (geoobjecttype 9)	468.097	GeoJSON de centenas de MB; inviável no navegador
Geometria dos setores	1,2 GB	Mesmo simplificada, muito pesada para enviar de uma vez
Tabela `value`	2,5 GB · 19 milhões de linhas	Organizada por setor (todos os indicadores juntos)
Ler 1 indicador para o Brasil	~200 s (disco frio)	As linhas do indicador estão espalhadas pela tabela inteira

A organização de `value` por setor é justamente o que torna os mapas de estado rápidos (todas as linhas de um setor ficam juntas no disco). O preço é que uma leitura "por indicador, no país todo" precisa tocar praticamente todas as páginas da tabela. Por isso o cálculo nacional tem de ser feito **antes**, e não a cada clique.

2. Visão geral da solução



Fluxo ao escolher Estado = Brasil

Não há botão "Exibir mapa": assim que setor estratégico, indicador e estado estão escolhidos, o mapa é carregado, e qualquer mudança nos campos o recarrega. Se uma carga ainda está em andamento quando o usuário muda um campo, ela é cancelada e descartada.

- O navegador baixa a lista de ids dos setores (uma vez por sessão, ~1 KB comprimida), a string de classes do indicador (~8 KB) e a legenda.
- Para cada setor com valor, aplica a classe como feature-state do MapLibre (468 mil chamadas, ~2 s no navegador).
- O MapLibre pede os tiles da área visível em `/api/tiles/{z}/{x}/{y}.pbf`. A API devolve o tile guardado em `webtests_tiles` ou, se ainda não existir, gera com PostGIS, guarda e devolve.
- A cor de cada polígono vem de uma expressão `match` sobre a classe, montada a partir das cores da legenda.

Com um estado ou município escolhido, o fluxo é o mesmo, mas em vez da string de classes o navegador baixa os valores da área (`/api/values`) e o retângulo para enquadrar o mapa (`/api/bounds`); os polígonos vêm dos mesmos tiles (seção 4). Os limites de estados e municípios vêm de `/api/boundaries/{z}/{x}/{y}.pbf` (seção 5).

3. Geometria: tiles vetoriais com PostGIS

Cada tile é um quadrado do mapa num nível de zoom (padrão XYZ). A consulta de um tile:

- seleciona os setores que tocam o tile usando o novo índice espacial `geoobject_geom_gist`;
- simplifica a geometria com tolerância de meio pixel de tela no zoom do tile `ST_Simplify`;

- converte para coordenadas do tile com `ST_AsMVTGeom` (grade de 4096 unidades, corte nas bordas com margem de 64);
- gera o tile no formato Mapbox Vector Tile com `ST_AsMVT`. O id de cada feição é o `geoobject.id`; não há outros atributos (nome e município são buscados ao passar o mouse), o que deixa os tiles cerca de 40% menores.

Setores menores que 2 pixels

No zoom do país, um setor urbano ocupa menos de um pixel. Se fosse desenhado como polígono, as cidades sumiriam (ou virariam milhares de polígonos invisíveis). Nesses casos o tile desenha **um quadrado de 2×2 pixels por célula**, com a cor do maior setor da célula. Por isso o mapa nacional tem aparência "em blocos"; ao aproximar o zoom os setores reais aparecem.

Os tiles não dependem do indicador: cada um é gerado uma única vez e guardado na tabela `webtests_tiles`. O navegador também os guarda por 1 dia (Cache-Control).

Tiles pré-calculados (zoom 2 a 9, área do Brasil)

Zoom	Tiles	Com dados	Total	Maior tile
2	2	2	0,2 MB	190 KB
3	4	3	0,7 MB	500 KB
4	9	7	1,9 MB	717 KB
5	20	15	3,4 MB	697 KB
6	64	40	5,3 MB	480 KB
7	225	124	9,0 MB	483 KB
8	870	442	15,9 MB	415 KB
9	3.420	1.632	25,5 MB	810 KB

Tiles sobre o oceano ou fora do Brasil ficam vazios e custam milissegundos. Zooms 10 a 12 cobrem áreas pequenas e são gerados sob demanda na primeira visita (3–15 s por tela, depois instantâneo). O zoom máximo dos tiles é 14; acima dele o mapa amplia os tiles do zoom 14. No zoom 14 a geometria não é simplificada (seção 4).

4. Mapas de estado e município também em tiles

Antes, o mapa de um estado buscava o GeoJSON de todos os setores, simplificado polígono a polígono com `ST_SimplifyPreserveTopology` (tolerância 0,0005° ≈ 55 m). Isso tinha dois problemas:

- **Falhas entre setores** ("slivers"): cada polígono era simplificado sozinho, então a borda comum a dois vizinhos ficava diferente em cada um, abrindo frestas visíveis no zoom de rua.
- **Lento para estados grandes**: Minas Gerais (51.387 setores) levava **45,5 s**, quase todo o tempo gerando e transferindo a geometria.

Agora estados e municípios usam a **mesma camada de tiles** do mapa nacional:

- `/api/values` devolve, para cada setor da área, o valor e a cor da legenda; o navegador aplica ambos como feature-state `{color, value}`. Setores fora da área ficam transparentes.
- `/api/bounds` devolve o retângulo da área (`ST_Extent` dos municípios, algumas centenas de polígonos em vez de dezenas de milhares de setores), usado para enquadrar o mapa. Fica em cache no servidor.
- No zoom máximo (14) a tolerância de simplificação é zero: só resta o arredondamento para a grade do tile, que é igual para os dois lados de uma borda comum. Resultado: **nenhuma fresta**, mesmo ampliando até o zoom 16.

- O popup continua mostrando o valor exato do setor (ex.: 0,5000), que vem de `/api/values`.

Tempo de carga

Área	Setores	Antes (GeoJSON)	Agora (tiles)
Minas Gerais	51.387	45,5 s	3,1–4,0 s
Belo Horizonte	5.166	—	0,6 s

O endpoint `/api/geometry` (GeoJSON) foi removido.

5. Limites de estados e municípios

Como as linhas entre setores foram removidas, os limites administrativos vêm de uma segunda camada de tiles, `/api/boundaries/{z}/{x}/{y}.pbf`, gerada do mesmo jeito que a dos setores, mas com linhas (`ST_Boundary`) em vez de polígonos:

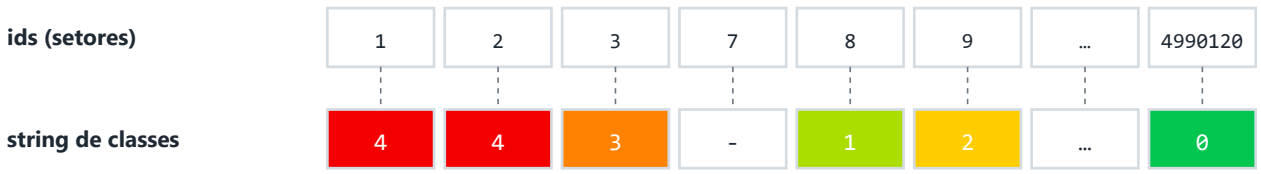
- camada `states`: limites dos 27 estados (geoobjecttype 4), em todos os zooms;
- camada `counties`: limites dos 5.570 municípios (geoobjecttype 1), a partir do zoom 6 (abaixo disso as linhas ficariam amontoadas);
- estilo: municípios em linhas brancas finas, estados em linhas escuras por cima dos setores.

A partir do zoom 11 os próprios setores também ganham um contorno branco fino. Ele não precisa de tiles novos: é uma camada de linhas sobre os mesmos tiles dos setores, só para os setores com cor (os de fora da área escolhida continuam invisíveis).

Os tiles de limites são pequenos (3–56 KB) e rápidos de gerar (0,1–0,6 s cada). Ficam na tabela `webtests_boundary_tiles`: 4.614 tiles dos zooms 2 a 9, **9 MB** no total (z6 0,7 MB · z7 1,3 MB · z8 2,2 MB · z9 3,7 MB). O custo para o navegador é desprezível comparado ao dos setores.

6. Cores: a “string de classes”

Para cada indicador, o servidor guarda um texto com um caractere por setor, na mesma ordem da lista de ids dos setores (crescente). O caractere é o índice da faixa da legenda (`0` a `9`) ou `-` quando o setor não tem valor:



posição i da string = legenda do i-ésimo setor; "-" = sem valor

- 468.097 caracteres ≈ 468 KB, que comprimem para **~8 KB** (há longas sequências iguais).
- A lista de ids vai codificada por diferenças (primeiro id, depois incrementos — quase todos iguais a 1): **~1 KB** comprimida.
- A string guarda a **versão** da lista de ids (hash). Se a lista mudar, o servidor recalcula e o navegador recarrega a lista.
- No navegador, cada classe vira `feature-state {c: classe}` do setor; a cor é escolhida pela expressão `['match', ['feature-state','c'], 0, cor0, 1, cor1, …]`.

Valor exato no popup

Como só a classe é pré-calculada, o popup do mapa nacional busca o valor exato do setor ao passar o mouse, junto com o nome: `/api/geoobject/{id}?indicator_id=` lê uma linha pelo índice (geoobject_id, indicator_id, year), em ~60–140 ms. A resposta fica em cache no navegador por setor e indicador; enquanto chega, o popup mostra a faixa da legenda.

Regra de classificação (correção de um erro)

As faixas da legenda são cadastradas como 0,00–0,19, 0,20–0,39, …, 0,80–1,00. A consulta antiga usava `minvalue ≤ valor < maxvalue`, que deixava de fora o valor **1,00** e os valores entre faixas (ex.: 0,195). No indicador "Balanço hídrico para agropecuária", **149.126 setores** com valor 1,00 ficavam invisíveis no mapa.

> A regra nova, usada no mapa nacional e também nos mapas de estado e município: **o valor recebe a faixa com o maior** `minvalue` **que seja ≤ valor**; valor ausente recebe a faixa sem limites ("Dado indisponível").

7. Pré-cálculo

Um único comando gera os três caches:

```
python -m lib.precompute              # classes, tiles dos setores e limites
python -m lib.precompute classes      # só as strings de classes
python -m lib.precompute tiles 9      # só os tiles dos setores, até o zoom 9
python -m lib.precompute boundaries   # só os tiles de limites
docker compose exec app python -m lib.precompute   # dentro do Docker
```

- **Strings de classes:** uma leitura sequencial de `value` agrupada por indicador gera os 44 indicadores do setor censitário de uma vez: **345 s no total**, contra ~200 s por indicador lendo um a um.
- **Tiles:** zooms 2 a 9 sobre a área do Brasil, 4 em paralelo. Os zooms baixos são os mais lentos (cada tile cobre boa parte do país: ~8–9 min no zoom 2/3); do zoom 6 em diante, com a geometria já em memória no banco, cada nível leva de segundos a poucos minutos. Os tiles de limites são bem mais rápidos (frações de segundo cada).
- O comando pula o que já existe: pode ser interrompido e executado de novo sem perder trabalho.
- Indicador sem string pré-calculada é calculado na primeira requisição (lento) e guardado; tile ausente é gerado na hora e guardado.

8. Desempenho medido

Operação	Tempo
Mapa do Brasil, primeira carga (ids + classes + legenda + tiles)	~5–8 s
String de classes de um indicador (do banco / da memória)	0,27 s / 0,03 s
Aplicar as cores de 468 mil setores no navegador	~2–3 s
Tile já guardado	~0,1 s
Tile novo, zoom de cidade (ex.: São Paulo, z12)	2–4 s
Ler um indicador do país diretamente de value (sem pré-cálculo)	~200 s
Pré-cálculo das 44 strings de classes	345 s
Mapa de estado (valores, AC), com a nova regra de legenda	0,2–0,5 s
Mapa de Minas Gerais (51 mil setores): antes / agora	45,5 s / 3,1–4,0 s
Mapa de Belo Horizonte (5 mil setores)	0,6 s
Tile de limites novo (estados + municípios)	0,1–0,6 s

Durante o pré-cálculo dos tiles o banco fica ocupado; nesse período a primeira carga de um estado chegou a 69 s (a geometria do estado disputava disco com o pré-cálculo).

9. Objetos criados no banco (adaptabrasil)

Objeto	Para quê	Tamanho
`geoobject_geom_gist`	Índice espacial (GiST) em geoobject.geom, usado pela consulta de cada tile	20 MB
`webtests_tiles`	Tiles dos setores: (z, x, y) → bytes do tile (4.689 tiles)	62 MB
`webtests_boundary_tiles`	Tiles dos limites de estados e municípios (4.614 tiles)	9 MB
`webtests_tract_classes`	String de classes por indicador + versão da lista de ids	~2 MB

As tabelas de cache são compartilhadas por todas as instâncias do aplicativo que usam o mesmo banco (a máquina local e o servidor 3.224.52.81): o trabalho é feito uma vez só.

Quando recalcular

- **Valores ou legendas mudaram:** `truncate adaptabrasil.webtests_tract_classes;` e depois `python -m lib.precompute classes`. Reinicie o aplicativo (as strings também ficam em memória).
- **Geometria dos setores mudou:** `truncate adaptabrasil.webtests_tiles;` e depois `python -m lib.precompute tiles`. Se a lista de setores mudou, as strings de classes são recalculadas automaticamente (versão diferente).
- **Limites de estados ou municípios mudaram:** `truncate adaptabrasil.webtests_boundary_tiles;` e depois `python -m lib.precompute boundaries`.

10. Limitações e próximos passos

- **Aparência em blocos no zoom do país**, consequência dos quadrados de 2 pixels. Alternativa: mostrar municípios (agregados) até o zoom ~8 e setores a partir daí.
- **Pontinhos escuros em zooms intermediários**, também vindos dos quadrados de 2 pixels de setores muito pequenos.
- **~2–3 s para aplicar as cores** no navegador (o mapa nacional leva ~5 s no total). Pode cair aplicando só aos setores dos tiles carregados.
- **Contorno dos setores só a partir do zoom 11**: abaixo disso muitos setores são quadrados de 2 pixels e o contorno viraria ruído; as divisões visíveis são os limites de estados e municípios (seção 5).
- **PMTiles:** se gerar tiles no banco ficar lento, os mesmos tiles podem ser exportados para um arquivo estático único (tippecanoe/PMTiles) servido sem consultar o banco.

11. Referência

Endpoints novos

Endpoint	Retorno
`GET /api/tiles/config`	Limites do Brasil e zoom mínimo/máximo dos tiles
`GET /api/brasil/ids`	Ids dos setores, crescentes, codificados por diferença (`version`, `deltas`)
`GET /api/brasil/classes?indicator_id=`	String de classes do indicador (`version`, `classes`)

Endpoint	Retorno
`GET /api/tiles/{z}/{x}/{y}.pbf`	Tile vetorial dos setores (camada `tracts`, id = geoobject)
`GET /api/boundaries/{z}/{x}/{y}.pbf`	Tile de limites (camadas `states` e `counties`)
`GET /api/bounds?state_id=&county_id=`	Retângulo do estado ou município, para enquadrar o mapa
`GET /api/geoobject/{id}?indicator_id=`	Nome e município de um setor; com o indicador, também o valor (popup)

Removido: `GET /api/geometry` (GeoJSON dos mapas de estado).

Arquivos

Arquivo	Conteúdo
`lib/brasil.py`	Tiles de setores e de limites (consultas MVT, cache), lista de ids, strings de classes, regra de legenda
`lib/precompute.py`	Pré-cálculo: `python -m lib.precompute`
`lib/db.py`	Nova regra de legenda nos mapas de estado; `fetch_bounds`; nome do setor para o popup
`main.py`	Endpoints novos
`static/app.js`	Opção Brasil; mapa carregado automaticamente; uma camada de tiles para todos os mapas; limites e contornos; cores por feature-state; popup

Com o Debug ligado, a requisição das classes mostra também a consulta SQL de um tile de exemplo, pronta para colar no DBeaver.